

Exercice 1 :

Monitor prod_Con

```
{ // début de moniteur
    char buffer [M][N] ; // M buffers de taille N
    int in[N], out[N]; // indices d'entrée et de sortie des buffers
    int count[M]; // nombre d'éléments dans chaque buffer
    condition vide[M], plein[M] ;

    void Deposer(char x){ // déposer un élément x dans les M buffers
        int i ;
        for(i=0 ; i<M ; i++){
            if ( count[i]== N) cwait(vide[i]);
            buffer[i][ in[i] ] = x;
            in[i] = (in[i] + 1) % N;
            count[i] ++;
            csignal( plein[i]);
        }
    }

    void Retirer(char *x, int i){ // retirer un élément du buffer,
        // i indice de consommateur
        if ( count[i] == 0) cwait(plein[i]);
        x* = buffer[i][ out[i] ];
        out[i] = (out[i] + 1) % N;
        count[i] --;
        csignal( vide[i]);
    }

} // code d'initialisation
int i;
for(i =0; i<M; i++){
    in[i] = 0;
    out[i] = 0;
    count[i] = 0;
}
}
} // fin de moniteur
```

Exercice 2:

1) Priorité aux rédacteurs, le nombre de lecteurs dans la zone de données est limité à M.

```
#define M 20
Monitor Lecteur_Redacteur
{  int nb_R ;    // nombre des rédacteurs qui veulent accéder à la base de données.
    int nb_Lec1 ; // nombre de lecteurs dans la zone de données.
    int nb_Lec2 ; // nombre de lecteurs qui ont demandé l'accès à la base de données
    condition lec_Ok, red_OK ;

void debut_lecture(){
    nb_Lec2 ++ ;
    if ( ( nb_R > 0 ) || ( nb_Lec2 > M ) ) {
        cwait(lec_Ok) ;
        nb_Lec1 ++ ;
        if ( nb_Lec1 < M) csignal(lec_Ok);
    } else
        nb_Lec1 ++ ;
}

void fin_lecture(){
    nb_Lec1 -- ;
    nb_Lec2 -- ;
    if (nb_Lec1 == 0 ) csignal(red_Ok);
    else
        if (nb_R == 0) csignal(lec_Ok);
}

void debut_ecriture(){
    nb_R ++;
    if ( ( nb_R>1) || ( nb_Lec1>0) ) cwait(red_Ok);
}

void fin_ecriture(){
    nb_R --;
    if (nb_R >0) csignal(red_Ok);
    else csignal(lec_Ok);
}
{ // code initialisation
    nb_Lec1 =0;
    nb_Lec2 =0;
    nb_R =0;
}
} // fin de moniteur
```

2) Priorité aux lecteurs, le nombre de lecteurs dans la zone de données est limité à M.

```
#define M 20
```

```
Monitor Lecteur_Redacteur
```

```
{ int red ; // red ==1 rédaction en cours
  int nb_Lec1 ; // nombre de lecteurs dans la base de données.
  int nb_Lec2 ; // nombre de lecteurs qui ont demandé l'accès à la base de données
  condition lec_Ok, red_OK ;
```

```
void debut_lecture(){
  nb_Lec2 ++ ;
  if ( ( red == 1 ) || ( nb_Lec2 > M ) ) {
    cwait(lec_Ok) ;
    nb_Lec1 ++ ;
    if ( nb_Lec1 < M) csignal(lec_Ok);
  } else
    nb_Lec1 ++ ;
}
```

```
void fin_lecture(){
  nb_Lec1 -- ;
  nb_Lec2 -- ;
  if (nb_Lec2 > nb_Lec1 ) csignal(lec_Ok);
  if (nb_Lec2 == 0) csignal(red_Ok);
}
```

```
void debut_ecriture(){
  if ( ( red == 1 ) || ( nb_Lec2 > 0 ) ) cwait(red_Ok);
  red =1;
}
```

```
void fin_ecriture(){
  red =0;
  if (nb_Lec2 > 0 ) csignal(lec_Ok);
  else csignal(red_Ok);
}
{ // code initialisation
  nb_Lec1 =0;
  nb_Lec2 =0;
  nb_R =0;
}
} // fin de moniteur
```

Exercice 3 :

Monitor BDD

```
{  
condition  ecrireOK , lireOK ;  
int  ne = 0 ;
```

void Ecriture () {

```
if (ne == M) cwait (ecrireOK);  
    <Ecrire dans la BDD>  
    ne ++ ;  
if (ne == M) csignal (lireOK);  
}
```

void Lecture () {

```
if (ne < M) cwait (lireOK);  
    <lire la BDD>  
    <imprimer>  
    ne = 0  
    csignal (ecrireOK);  
}
```

```
{          // code d'initialisation
```

```
    ne = 0 ;
```

```
}
```

```
} // fin moniteur
```

Processus Ri () {

```
    BDD.ecriture ( );
```

```
}
```

Processus L () {

```
    while(1){  
        BDD.Lecture ( );  
    }
```

```
}
```

int main () {

```
    parbegin(L(), D1(),D2(),..., DN() );
```

```
    return 0 ;
```

```
}
```